

claude-windows / dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3\subagents\workflows\wf_b0e273c7-38a\agent-ae0abed9d68b6fc48.jsonl`
- SHA-256: `8d3cd6c5783f29b537e31cf84e9e1c504b2da0c67f6481663c63564997f4780c`
- Source modified: `2026-06-21T08:01:30+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_b0e273c7-38a`
- session_id: `dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3`
```

Transcript

1. user (2026-06-21T07:56:51.760Z)

You are reviewing milestone M2 of COMPUTE_DECOUPLED_SERVING in the repo at C:/Users/avidu/Projects/badminton-highlight-indexer.
Spec: docs/COMPUTE_DECOUPLED_SERVING/README.md \$7 row M2 = "input_backend/input_root; wire main CLI/API input through StorageRef.parse_uri (local=identity); storage-fake tests". M2 must be SAFE and default-OFF: with the default config (storage.input_backend='local', input_root=''), the loaded input path must be byte-identical to before ? a bare/local path is read in place with NO copy. A bucket/Drive URI (gs://, s3://, gdrive://) is fetched to a local scratch file before processing.

The change (read all of these + git diff against origin/master):

- backend/config/models.py: StorageConfig gains input_backend + input_root.
- backend/storage/ref.py: resolve_input_uri() (precedence: explicit scheme > anchored local path > input_root join > input_backend prefix > bare-local).
- backend/storage/__init__.py: acquire_local_input() (local=identity, remote=mkdtemp+fetch) + input_is_local().
- backend/main.py: _execute_processing() (API/job path) + the CLI --video-path block both acquire-then-use the local path (keeping the original URI for the DB record + report); /api/process skips the local os.path.exists() gate for a non-local URI.
- tests: tests/test_input_source.py (new) + 3 tests in tests/test_api_endpoints.py.

Reference the worker's already-shipped pattern (backend/worker/__main__.py:91-105 cmd_infer) and the path-safety helpers (backend/utils/paths.py: validate_input_path / allowed_video_root). Ground EVERY claim in code with file:line. Give the exact triggering input + wrong-vs-right behaviour. Prefer FEWER, HIGH-CONFIDENCE findings. A clean bill is a valid, good result.

LENS: security & path-safety. /api/process now SKIPS os.path.exists for a non-local URI (main.py around the validate_input_path block). (a) Can a crafted video_path bypass validate_input_path's null-byte check or the allowed_video_root containment in HOSTED mode (allowed_root set)? Trace what input_is_local + resolve_input_uri return for adversarial inputs (e.g. "gs://.../etc", "local://.../x", " gs://x", "C:...\x", "" empty, a URL with embedded creds). (b) In hosted mode does a non-local URI get correctly rejected (resolve_under_root realpath on a "gs://.." string)? (c) Does acquire_local_input fetch to a safe scratch (tempfile.mkdtemp) with a safe filename (ref.name) ?

any
path-traversal via a key like "../../etc/passwd"? (d) SSRF/unexpected-egress: can a user URI cause an
unintended network fetch on a deployment that didn't opt in? Is that gated by config?

2. assistant (2026-06-21T07:56:54.562Z)

I'll review milestone M2 of COMPUTE_DECOUPLED_SERVING with a focus on security & path-safety.
Let me start by reading the spec and the relevant code.

3. user (2026-06-21T07:56:58.316Z)

```
---BRANCH---
feat/serving-m1-deployment-profile
---LOG---
9c64ac5 docs(serving): mark $7 tracker M1 ? + changelog (PR #279)
368155f feat(serving): M1 ? deployment.profile + compute_target config (pure, additive, default-OFF)
f518e97 docs(serving): M0 APPROVED + lock D13-D15 (switchable profiles, Drive Northstar, consent-not-surprise) (#278)
49c082a docs(serving): lock the 5 M0 gating decisions (D7-D12) + add the data-flywheel harness (M11) (#277)
f592281 docs: point I18N_PLAN + UI_PROPOSAL (+ DOC_STATUS) at the khelsutra UX/l10n project (#275)
```

4. user (2026-06-21T07:56:58.557Z)

```
backend/config/__init__.py      | 5 +
backend/config/loader.py        | 100 ++++++
backend/config/models.py        | 21 ++
backend/config/presets.py       | 120 ++++++
docs/COMPUTE_DECOUPLED_SERVING/README.md | 3 +-
tests/test_config_deployment.py | 371 ++++++
6 files changed, 617 insertions(+), 3 deletions(-)
```

5. assistant (2026-06-21T07:57:01.807Z)

The diff against origin/master shows M1 (deployment profile) changes, not M2. The M2 changes (storage/ref.py, storage/__init__.py, main.py input wiring) aren't in this branch yet. Let me check the actual state ? M2 may be uncommitted or on a different branch.

6. user (2026-06-21T07:57:03.565Z)

```
---STATUS---
M backend/config/models.py
M backend/main.py
M backend/storage/__init__.py
M backend/storage/ref.py
M tests/test_api_endpoints.py
?? artifacts/
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
?? tests/test_input_source.py
---ALL BRANCHES---
+ bhi-execpolicy
  chore/consolidate-fusion-schema-predicate
  docs/adr-012-gpu-cloudrun
  docs/archive-decoupled-compute
  docs/archive-decoupled-v2
  docs/compute-decoupled-serving
  docs/court-floor-rally-end
  docs/decouple-scope-defer
  docs/deploy-report-polish
  docs/desktop-app-plan
  docs/doc-status-register
```

```
docs/drop-digitalocean-gcp-only
docs/findings-1-2-pervideo-nextsteps
docs/gcp-deploy-report
docs/gcp-pivot-runbook
docs/golden-regression-fixtures-plan
docs/lock-m0-decisions
+ docs/lovo-resweep-n13
  docs/m0-approved-d13-d15
  docs/m2-finalized-report
  docs/point-i18n-ui-to-khelsutra
+ docs/rally-gap-closure
+ docs/recording-single-match
  docs/restructure-field-roora-data
+ dryrun/cuj-local
  feat/api-m1-health-downloadurl
  feat/gdrive-storage-backend
  feat/highlights-runner
  feat/local-no-ai-segmenter
  feat/long-rally-lens
```

7. user (2026-06-21T07:57:04.452Z)

```
__init__.py
__pycache__
base.py
gcs.py
gdrive.py
local.py
ref.py
s3.py
---REF---
-rw-r--r-- 1 avidu 197609 4455 Jun 21 13:23 backend/storage/ref.py
```

8. assistant (2026-06-21T07:57:07.281Z)

The M2 changes are uncommitted in the working tree (modified files + new test). Let me look at the actual diff against origin/master for the M2 files specifically.

9. user (2026-06-21T07:57:07.971Z)

```
diff --git a/backend/config/models.py b/backend/config/models.py
index 3612bd8..a31c9da 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -302,6 +302,16 @@ class StorageConfig(BaseModel):
     backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
     # Output root for produced artifacts. "" => fall back to output.output_dir (non-breaking).
     output_root: str = ""
+
+ # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
+ `backend`/`output_root`).
+ # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is read IN
+ PLACE
+ # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
+ bare/relative
+ # input name is resolved against ``input_root`` (e.g. ``input_root="gs://bucket/videos"`` +
+ "x.mp4"
+ # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
+ ``input_backend``
+ # (``"gs"`` + "bucket/x.mp4" => ``gs://bucket/x.mp4``). An explicit scheme on the input
+ # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs are
+ fetched to a
+ # scratch dir before processing (see ``backend.storage.acquire_local_input``).
+ input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
+ input_root: str = ""
```

```

# --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
# One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
# so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the root
diff --git a/backend/main.py b/backend/main.py
index 942217e..33c7995 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -32,6 +32,7 @@ from backend.utils.paths import resolve_under_root, safe_output_filename,
valida
from backend.config import load_config as load_app_config, AppConfig, StitchingConfig # new
typed config
from backend.results import Failure # standardized error/result contract
from backend.reporting import emit_indexer_report
+from backend import storage # M2: URI-aware input acquisition (local = identity passthrough)

app = FastAPI(title="Sports Highlight Indexer API")

@@ -151,13 +152,18 @@ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:
    notify = progress or (lambda stage: None)

    notify("hashing")
-    video_id = compute_video_id(req.video_path)
+    # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged (identity
?
+    # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The original
+    # req.video_path (the source URI) is kept for the DB record + report; only the file-reading
+    # consumers (hash / validators / segmenter) use the resolved local path.
+    local_video = storage.acquire_local_input(req.video_path, getattr(global_config, "storage",
None))
+    video_id = compute_video_id(local_video)
    was_reingest = db_instance.get_video(video_id) is not None

    # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the report)
    notify("validating")
    logger.info(f"Running validations for {req.video_path}...")
-    val_results, val_context = registry.run_all_with_context(req.video_path, global_config)
+    val_results, val_context = registry.run_all_with_context(local_video, global_config)
    failures = [r for r in val_results if not r.passed]

    # typed AppConfig: attribute access for the default segmenter (with safe fallback)
@@ -205,7 +211,7 @@ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:
    play_wm, cand_wm = db_instance.segment_watermarks(video_id)
    segmenter = segmenter_cls(db_instance, global_config)
    try:
-        result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
+        result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
    except Exception:
        # A raising segmenter must not leave half-written rows: restore the pre-run
        # state (same destroy-after-confirm contract as the Failure branch), then let
@@ -265,13 +271,13 @@ def process_video(req: ProcessRequest):
    validate_input_path(req.video_path, allowed_root=allowed_root)
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e)) from e
-    if not os.path.exists(req.video_path):
-        raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
+    # M2: the local-existence fast-fail only applies to a LOCAL input. A bucket/Drive URI can't
be
+    # cheaply os.path.exists()'d ? its existence is verified when the worker fetches it (a
fetch miss

```

```

+ # fails the job loudly). validate_input_path above still runs for every input: its null-
byte guard
+ # always applies, and a configured allowed_video_root (hosted mode) rejects a non-local
URI.
+ if storage.input_is_local(req.video_path, getattr(global_config, "storage", None)):
+     if not os.path.exists(req.video_path):
+         raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
+     if req.segmenter and not segmenter_registry.get(req.segmenter):
+         available = list(segmenter_registry.list_available().keys())
+         raise HTTPException(
@@ -577,16 +588,20 @@ def main():
+     # CLI processing mode (no web UI needed)
+     if args.video_path and not args.server:
+         print(f"[CLI] Processing video file: {args.video_path}")
-         if not os.path.exists(args.video_path):
+         # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is fetched to
scratch.
+         # The local-existence fast-fail only applies to a local input; a remote fetch fails
loudly.
+         storage_cfg = getattr(global_config, "storage", None)
+         if storage.input_is_local(args.video_path, storage_cfg) and not
os.path.exists(args.video_path):
+             print(f"[FAIL] Video file not found: {args.video_path}")
+             return
+         local_video = storage.acquire_local_input(args.video_path, storage_cfg)

-         video_id = compute_video_id(args.video_path)
+         video_id = compute_video_id(local_video)
+         was_reingest = db_instance.get_video(video_id) is not None

+         # Validate (with-context variant surfaces ffprobe_meta for the report)
+         print("[CLI] Validating...")
-         val_results, val_context = registry.run_all_with_context(args.video_path,
global_config)
+         val_results, val_context = registry.run_all_with_context(local_video, global_config)
+         failures = [r for r in val_results if not r.passed]

+         # Save video status
@@ -619,7 +634,7 @@ def main():
+         segmenter_actual = seg_name
+         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
+         segmenter = segmenter_cls(db_instance, global_config)
-         result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
+         result = segmenter.process_video(video_id, local_video, max_frames=args.max_frames)
+         segmentation_ran = True

+         if isinstance(result, Failure):
diff --git a/backend/storage/__init__.py b/backend/storage/__init__.py
index 78b4795..656274f 100644
--- a/backend/storage/__init__.py
+++ b/backend/storage/__init__.py
@@ -13,17 +13,58 @@ passthrough. See ``docs/DECOUPLED_COMPUTE/``.
+ from __future__ import annotations

+ import os
+import tempfile
+ from typing import Any, Dict, Iterator, Optional

+ from backend.storage.base import StorageBackend
- from backend.storage.ref import StorageRef, StorageStat, parse_uri
+from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri

+__all__ = [

```

```

- "StorageBackend", "StorageRef", "StorageStat", "parse_uri",
+ "StorageBackend", "StorageRef", "StorageStat", "parse_uri", "resolve_input_uri",
  "get_storage", "fetch", "put", "list_uris",
+ "acquire_local_input", "input_is_local",
]

+def _storage_dict(storage_config: Any) -> Dict[str, Any]:
+    """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
+    if storage_config is None:
+        return {}
+    if hasattr(storage_config, "model_dump"):
+        return storage_config.model_dump() # type: ignore[no-any-return]
+    return dict(storage_config)
+
+
+def input_is_local(raw: str, storage_config: Any = None) -> bool:
+    """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ? i.e. a
+    plain on-disk file the caller can stat/validate directly. False for a bucket / Drive URI
+    that must be fetched first. Lets callers keep their local-only checks (os.path.exists,
+    allowed_video_root) for the local case and skip them for a remote source."""
+    sc = _storage_dict(storage_config)
+    uri = resolve_input_uri(raw, input_backend=sc.get("input_backend", "local"),
+                           input_root=sc.get("input_root", ""))
+    return parse_uri(uri).backend == "local"
+
+
+def acquire_local_input(raw: str, storage_config: Any = None, *,
+                        scratch_parent: "str | None" = None) -> str:
+    """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for processing.
+
+    Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
+    behaviour
+    byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a bare name
+    under a
+    non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir and the
+    local
+    path returned. The scratch copy is left in place (a pure intermediate, regenerable from the
+    source) ? same as the worker's ``cmd_infer`` pull, so cache-hygiene tooling reclaims it."""
+    sc = _storage_dict(storage_config)
+    uri = resolve_input_uri(raw, input_backend=sc.get("input_backend", "local"),
+                           input_root=sc.get("input_root", ""))
+    ref = parse_uri(uri)
+    if ref.backend == "local":
+        return ref.key # identity ? no temp, no copy, byte-identical to passing the path
+    directly
+    scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
+    return fetch(uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
+
+
+def _backend_class(name: str):
+    """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
+    if name == "local":
diff --git a/backend/storage/ref.py b/backend/storage/ref.py
index ad5d784..60e2430 100644
--- a/backend/storage/ref.py
+++ b/backend/storage/ref.py
@@ -66,3 +66,33 @@ def parse_uri(uri: str) -> StorageRef:
     bucket, _, key = rest.partition("/")
     return StorageRef(scheme, bucket, key, s)
     raise ValueError(f"unsupported storage URI scheme: {scheme}:// (in {s!r})")
+
+
+def resolve_input_uri(raw: str, *, input_backend: str = "local", input_root: str = "") -> str:
+    """Resolve a user-supplied input reference into a storage URI (COMPUTE_DECOUPLED_SERVING

```

```

M2).
+
+     Precedence (first match wins):
+     1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``, ``local://``,
+ ``gdrive://``) ? as-is.
+     2. An ABSOLUTE local path (``/data/x.mp4``, ``C:\\?``, ``C:/?``) ? as-is (always local).
+     3. A bare/relative name + a non-empty ``input_root`` ? joined under it
+     (``"gs://bucket/videos"`` + ``"x.mp4"`` ? ``"gs://bucket/videos/x.mp4"``).
+     4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``"gcs"`` +
+ ``"bucket/x"`` ?
+     ``"gcs://bucket/x"``).
+     5. Otherwise ? ``raw`` unchanged (a relative/bare LOCAL path ? today's behaviour).
+
+     So the default (``input_backend="local"`` , ``input_root=""``) is an identity passthrough
for every
+     path used today, preserving byte-for-byte behaviour.
+     """
+
+     s = str(raw)
+     if _SCHEME_RE.match(s):          # 1. explicit scheme wins
+         return s
+
+     # 2. an anchored LOCAL path is always local, never joined. Detect this OS-INDEPENDENTLY ?
+     #     os.path.isabs is platform- AND Python-version-dependent (on Windows + py3.13 a POSIX-
style
+     #     "/data/x" is NOT absolute). Anchored == a leading "/" or "\\ ", or a Windows drive
prefix "X:".
+     if s.startswith((" /", "\\ ")) or (len(s) >= 2 and s[1] == ":" and s[0].isalpha()):
+         return s
+
+     if input_root:                   # 3. join a relative name under the configured root
+         return input_root.rstrip("/") + "/" + s.lstrip("/")
+
+     if input_backend and input_backend != "local":  # 4. prefix a bucket/key with the declared
backend
+         return f"{input_backend}://{s}"
+
+     return s                          # 5. bare/relative local path ? unchanged

```

10. assistant (2026-06-21T07:57:10.687Z)

Now let me read the full files for context ? particularly `ref.py` (the `\_SCHEME\_RE` and `parse\_uri`), `paths.py` (validate_input_path / allowed_video_root / resolve_under_root), and the worker's reference pattern.

11. user (2026-06-21T07:57:11.671Z)

```

1     """Storage value objects + URI parsing for the pluggable storage layer.
2
3     A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5         <bare path>                -> local    (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6         local://<path>              -> local    (local:///abs or local://rel/p)
7         s3://<bucket>/<key>         -> s3        (AWS + R2 + DO Spaces + MinIO via
endpoint_url)
8         gcs://<bucket>/<key>       -> gcs        (gs:// accepted, normalized to gcs://)
9         gdrive://<path>            -> gdrive    (path under your locally-MOUNTED Google
Drive "My Drive")
10
11     No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12     stays cheap.
13     """
14     from __future__ import annotations
15
16     import re
17     from dataclasses import dataclass
18     from datetime import datetime
19     from typing import Optional

```

```

20
21 _SCHEME_RE = re.compile(r"^[A-Za-z][A-Za-z0-9+.\-]*://")
22 _KNOWN = ("local", "s3", "gcs", "gdrive")
23
24
25 @dataclass(frozen=True)
26 class StorageStat:
27     """Lightweight object metadata (uniform across backends)."""
28     size: int
29     modified: Optional[datetime] = None
30     etag: Optional[str] = None
31     content_type: Optional[str] = None
32
33
34 @dataclass(frozen=True)
35 class StorageRef:
36     """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and
37     ``gdrive`` it is
38     empty and ``key`` carries the path (a filesystem path, or a path under the mounted
39     Drive)."""
40     backend: str
41     bucket: str
42     key: str
43     uri: str
44
45     @property
46     def name(self) -> str:
47         return self.key.rstrip("/").rsplit("/", 1)[-1]
48
49 def parse_uri(uri: str) -> StorageRef:
50     """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.
51
52     A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
53     have no ``//``) is treated as a local filesystem path ? preserving today's
54     behaviour.
55     """
56     s = str(uri)
57     m = _SCHEME_RE.match(s)
58     if not m:
59         return StorageRef("local", "", s, f"local://{s}")
60     scheme = m.group(1).lower()
61     rest = s[m.end():]
62     if scheme == "gs":
63         scheme = "gcs"
64     if scheme in ("local", "gdrive"):
65         # Path-keyed, no bucket: local FS, or a path under the mounted Google Drive "My
66         Drive".
67         return StorageRef(scheme, "", rest, s)
68     if scheme in _KNOWN:
69         bucket, _, key = rest.partition("/")
70         return StorageRef(scheme, bucket, key, s)
71     raise ValueError(f"unsupported storage URI scheme: {scheme}:// (in {s!r})")
72
73 def resolve_input_uri(raw: str, *, input_backend: str = "local", input_root: str = "")
74 -> str:
75     """Resolve a user-supplied input reference into a storage URI
76     (COMPUTE_DECOUPLED_SERVING M2).
77
78     Precedence (first match wins):
79     1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``, ``local://``,
80     ``gdrive://``) ? as-is.
81     2. An ABSOLUTE local path (``/data/x.mp4``, ``C:\\?``, ``C:/?``) ? as-is (always
82     local).

```

```

77         3. A bare/relative name + a non-empty ``input_root`` ? joined under it
78         (``"gs://bucket/videos"`` + ``"x.mp4"`` ? ``"gs://bucket/videos/x.mp4"``).
79         4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``"gcs"`` +
80         ``"bucket/x"`` ?
81         ``"gcs://bucket/x"``).
82         5. Otherwise ? ``raw`` unchanged (a relative/bare LOCAL path ? today's behaviour).
83
84     So the default (``input_backend="local"`` , ``input_root=""``) is an identity
85     passthrough for every
86     path used today, preserving byte-for-byte behaviour.
87     """
88     s = str(raw)
89     if _SCHEME_RE.match(s):          # 1. explicit scheme wins
90         return s
91     # 2. an anchored LOCAL path is always local, never joined. Detect this OS-
92     INDEPENDENTLY ?
93     #     os.path.isabs is platform- AND Python-version-dependent (on Windows + py3.13 a
94     POSIX-style
95     #     "/data/x" is NOT absolute). Anchored == a leading "/" or "\\ ", or a Windows
96     drive prefix "X:".
97     if s.startswith((" /", "\\ ")) or (len(s) >= 2 and s[1] == ":" and s[0].isalpha()):
98         return s
99     if input_root:                  # 3. join a relative name under the configured
100     root
101         return input_root.rstrip("/") + "/" + s.lstrip("/")
102     if input_backend and input_backend != "local":  # 4. prefix a bucket/key with the
103     declared backend
104         return f"{input_backend}://{s}"
105     return s                        # 5. bare/relative local path ? unchanged

```

12. user (2026-06-21T07:57:11.730Z)

```

1     """Path-safety helpers for the API boundary (untrusted request fields)."""
2     import os
3     from typing import Optional
4
5
6     def safe_output_filename(name: str) -> str:
7         """Validate an output filename is a BARE name (no dir part, no traversal, not
8         absolute).
9         `os.path.join(output_dir, name)` silently discards `output_dir` when `name` is
10         absolute,
11         and `..` segments escape it ? so an unchecked `output_filename` is an arbitrary-
12         write
13         primitive. Returns the name unchanged when safe; raises ValueError otherwise.
14         """
15         if not name or not name.strip():
16             raise ValueError("output_filename must not be empty")
17         if "\x00" in name:
18             raise ValueError("output_filename must not contain a null byte")
19         if "/" in name or "\\ " in name:
20             raise ValueError(f"output_filename must not contain a path separator: {name!r}")
21         if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
22             raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
23         return name
24
25     def resolve_under_root(path: str, root: str) -> str:
26         """Return the real path, asserting it is contained under `root` (raises ValueError
27         if not)."""
28         rp = os.path.realpath(path)
29         rr = os.path.realpath(root)
30         try:

```

```

29         contained = os.path.commonpath([rp, rr]) == rr
30     except ValueError: # different drives on Windows
31         contained = False
32     if not contained:
33         raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
34     return rp
35
36
37 def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
38     """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.
39
40     With no allowed_root (default, single-user local mode) only the null-byte check
41     applies,
42     so the local 'point it at any of my videos' workflow is preserved. Setting an
43     allowed_root (a future hosted/tenant deployment) enforces containment.
44     """
45     if "\\x00" in path:
46         raise ValueError("video_path must not contain a null byte")
47     if allowed_root:
48         resolve_under_root(path, allowed_root)

```

13. assistant (2026-06-21T07:57:14.329Z)

Now let me look at the order of operations in `process\_video` ? specifically whether `validate\_input\_path` runs before the existence check, and the worker's reference pattern.

14. user (2026-06-21T07:57:15.289Z)

```

250         paths = emit_indexer_report(
251             db=db_instance, video_id=video_id, video_path=req.video_path,
252             config=global_config,
253             val_results=val_results, val_context=val_context,
254             segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
255             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
256         )
257         if paths and isinstance(response, dict):
258             response["reports"] = paths
259
260 @app.post("/api/process", status_code=202)
261 def process_video(req: ProcessRequest):
262     """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
263
264     Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
265     client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
266     runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
267     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
268     """
269     allowed_root = getattr(getattr(global_config, "security", None),
270 "allowed_video_root", None)
271     try:
272         validate_input_path(req.video_path, allowed_root=allowed_root)
273     except ValueError as e:
274         raise HTTPException(status_code=400, detail=str(e)) from e
275     # M2: the local-existence fast-fail only applies to a LOCAL input. A bucket/Drive
276     # URI can't be
277     # cheaply os.path.exists()'d ? its existence is verified when the worker fetches it
278     # (a fetch miss
279     # fails the job loudly). validate_input_path above still runs for every input: its
280     # null-byte guard
281     # always applies, and a configured allowed_video_root (hosted mode) rejects a non-
282     # local URI.
283     if storage.input_is_local(req.video_path, getattr(global_config, "storage", None)):
284         if not os.path.exists(req.video_path):

```

```

280         raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}''")
281     if req.segmenter and not segmenter_registry.get(req.segmenter):
282         available = list(segmenter_registry.list_available().keys())
283         raise HTTPException(
284             status_code=400,
285             detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
286         )
287
288     job_id = uuid.uuid4().hex
289     if not db_instance.create_job(job_id, req.video_path, req.segmenter,
290                                 req.player_pool, req.max_frames):
291         raise HTTPException(status_code=500, detail="Failed to persist job record.")
292     _get_executor().submit(_drain_due_jobs)
293     return JSONResponse(
294         status_code=202,

```

15. user (2026-06-21T07:57:16.527Z)

```

80     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
81         with open(path, "w") as f:
82             json.dump({
83                 "video_id": video_id,
84                 "status": status,
85                 "segment_count": len(segments),
86                 "segments": [{"start": float(s.get("start_time", 0.0)),
87                             "end": float(s.get("end_time", 0.0))} for s in segments],
88             }, f, indent=2)
89
90
91     def cmd_infer(args: argparse.Namespace) -> int:
92         config = load_config(args.config) if args.config else load_config()
93         _apply_overrides(config, args.set)
94         storage_cfg = config.storage.model_dump()
95
96         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
97         os.makedirs(scratch, exist_ok=True)
98         config.output.output_dir = scratch
99
100        # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
101        local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
102        print(f"[worker] pulled {args.video_uri} -> {local_video}")
103
104        # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
105        video_id = compute_video_id(local_video)
106
107        # 3. VALIDATE (same registry as the main CLI).
108        val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
109        failures = [r for r in val_results if not r.passed]
110        db = Database(os.path.join(scratch, config.output.db_name))
111        db.add_video(video_id, local_video, "failed" if failures else "processed",
[r.model_dump() for r in val_results])
112
113
114        segments: List[dict] = []

```

16. assistant (2026-06-21T07:57:20.008Z)

Now let me look at the `local.py` fetch implementation (what happens for a `local://` URI fetch) and the new test file to understand the intended behavior.

17. user (2026-06-21T07:57:20.900Z)

```
1      """Local-filesystem storage backend ? the default, zero new dependencies.
2
3      Wraps today's behaviour byte-for-byte: ``fetch_to_local`` is an identity passthrough
(the
4      file is already local), ``put`` is a copy (a no-op when src == dst). With
``backend='local'``
5      the whole pipeline behaves exactly as before the storage layer existed.
6      """
7      from __future__ import annotations
8
9      import os
10     import shutil
11     from datetime import datetime
12     from typing import Iterator
13
14     from backend.storage.base import StorageBackend
15     from backend.storage.ref import StorageRef, StorageStat
16
17
18     class LocalStorage(StorageBackend):
19         def __init__(self, *, base_dir: str = "", **_ignored) -> None:
20             # Relative keys resolve under base_dir when set; absolute keys are used as-is.
21             self.base_dir = base_dir
22
23         def _path(self, ref: StorageRef) -> str:
24             p = ref.key
25             if self.base_dir and not os.path.isabs(p):
26                 return os.path.join(self.base_dir, p)
27             return p
28
29         def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
30             # The object is already on the local FS ? return its path (identity). ``dst`` is
31             # honoured only if it names a *different* location (a copy is then made).
32             src = self._path(ref)
33             if dst is None or os.path.abspath(str(dst)) == os.path.abspath(src):
34                 return src
35             dst = str(dst)
36             if os.path.exists(dst) and not overwrite:
37                 return dst
38             os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
39             shutil.copyfile(src, dst)
40             return dst
41
42         def put(self, local_path, ref, *, content_type=None) -> StorageRef:
43             dst = self._path(ref)
44             if os.path.abspath(str(local_path)) != os.path.abspath(dst):
45                 os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
46                 shutil.copyfile(str(local_path), dst)
47             return ref
48
49         def exists(self, ref) -> bool:
50             return os.path.exists(self._path(ref))
51
52         def stat(self, ref) -> StorageStat:
53             st = os.stat(self._path(ref))
54             return StorageStat(size=st.st_size,
modified=datetime.fromtimestamp(st.st_mtime))
55
56         def list(self, prefix) -> Iterator[StorageRef]:
57             base = self._path(prefix)
58             if os.path.isdir(base):
59                 for root, _dirs, files in os.walk(base):
60                     for fn in sorted(files):
```

```

61         full = os.path.join(root, fn)
62         yield StorageRef("local", "", full, f"local://{full}")
63     elif os.path.exists(base):
64         yield StorageRef("local", "", base, f"local://{base}")
65
66     def delete(self, ref) -> None:
67         path = self._path(ref)
68         if os.path.exists(path):
69             os.remove(path)
70

```

18. user (2026-06-21T07:57:21.349Z)

```

1      """Input acquisition for COMPUTE_DECOUPLED_SERVING M2 ? resolve_input_uri /
input_is_local /
2      acquire_local_input.
3
4      The hard guarantee: the default config (input_backend='local', input_root='') is a byte-
for-byte
5      identity passthrough for every input path used today (no fetch, no copy). A bucket/Drive
source is
6      routed through storage.fetch to a local scratch file. All offline (the remote backend is
a
7      monkeypatched fetch ? no boto3/google-cloud-storage, no network).
8
9      OS note: os.path.isabs is platform-dependent, so the absolute-path cases use a POSIX-
style
10     '/data/...' (absolute on BOTH Windows and POSIX) and Windows backslash paths only with
input_root=''
11     (branch 5, OS-independent) ? the full suite runs cross-OS in CI.
12     """
13     import os
14
15     import pytest
16
17     from backend import storage
18     from backend.storage import acquire_local_input, input_is_local
19     from backend.storage.ref import resolve_input_uri
20
21
22     # ----- resolve_input_uri
23     @pytest.mark.parametrize("raw,ib,ir,expected", [
24         # 1. explicit scheme always wins (even over a non-local backend + root)
25         ("gs://b/x.mp4", "local", "", "gs://b/x.mp4"),
26         ("s3://b/x.mp4", "gcs", "gs://r", "s3://b/x.mp4"),
27         ("local://abs/x.mp4", "gcs", "gs://r", "local://abs/x.mp4"),
28         ("gdrive://Sharing/x.mp4", "local", "", "gdrive://Sharing/x.mp4"),
29         # 2. an anchored local path (leading slash, or drive prefix) is always local, never
joined ?
30         # OS-independently (not via os.path.isabs, which differs on Windows+py3.13)
31         ("/data/x.mp4", "gcs", "gs://r", "/data/x.mp4"),
32         ("\\\\server\\share\\x.mp4", "gcs", "gs://r", "\\\\server\\share\\x.mp4"),
33         ("C:\\v\\x.mp4", "gcs", "gs://r", "C:\\v\\x.mp4"),
34         ("C:/v/x.mp4", "gcs", "gs://r", "C:/v/x.mp4"),
35         # 3. a bare/relative name joins under a non-empty input_root
36         ("x.mp4", "local", "gs://bucket/videos", "gs://bucket/videos/x.mp4"),
37         ("sub/x.mp4", "local", "gs://bucket/videos/", "gs://bucket/videos/sub/x.mp4"), #
root trailing slash
38         # 4. input_backend prefixes a bare bucket/key when no input_root
39         ("bucket/x.mp4", "gcs", "", "gs://bucket/x.mp4"),
40         ("bucket/x.mp4", "s3", "", "s3://bucket/x.mp4"),
41         # 5. bare/relative local path ? today's behaviour, unchanged
42         ("x.mp4", "local", "", "x.mp4"),
43         ("rel/sub/x.mp4", "local", "", "rel/sub/x.mp4"),
44         ("C:\\v\\x.mp4", "local", "", "C:\\v\\x.mp4"), # Windows path, default config:

```

```

unchanged (OS-independent)
45     ])
46     def test_resolve_input_uri(raw, ib, ir, expected):
47         assert resolve_input_uri(raw, input_backend=ib, input_root=ir) == expected
48
49
50     def test_resolve_input_uri_default_is_identity():
51         for p in ("/data/clip.mp4", "rel/clip.mp4", "clip.mp4", "gs://b/k", "local://x"):
52             assert resolve_input_uri(p) == p
53
54
55     # ----- input_is_local
56     def test_input_is_local_true_for_local_paths():
57         assert input_is_local("/data/x.mp4") is True
58         assert input_is_local("rel/x.mp4") is True
59         assert input_is_local("local://x.mp4") is True
60
61
62     def test_input_is_local_false_for_buckets():
63         assert input_is_local("gs://b/x.mp4") is False
64         assert input_is_local("s3://b/x.mp4") is False
65         assert input_is_local("gdrive://Sharing/x.mp4") is False
66
67
68     def test_input_is_local_follows_input_root():
69         cfg = {"input_backend": "local", "input_root": "gs://bucket/videos"}
70         assert input_is_local("x.mp4", cfg) is False          # bare name resolves into the
bucket
71         assert input_is_local("/abs/x.mp4", cfg) is True      # absolute path stays local
72         assert input_is_local("gs://other/x.mp4", cfg) is False
73
74
75     def test_input_is_local_accepts_pydantic_storage_config():
76         from backend.config.models import StorageConfig
77         sc = StorageConfig(input_root="gs://bucket/videos")
78         assert input_is_local("x.mp4", sc) is False
79         assert input_is_local("/abs/x.mp4", sc) is True
80
81
82     # ----- acquire_local_input
83     def test_acquire_local_input_local_identity(tmp_path):
84         p = tmp_path / "v.mp4"
85         p.write_bytes(b"abc")
86         # Identity passthrough ? returns the SAME string, no copy, no temp dir.
87         assert acquire_local_input(str(p)) == str(p)
88
89
90     def test_acquire_local_input_relative_local_identity():
91         assert acquire_local_input("rel/clip.mp4") == "rel/clip.mp4"
92
93
94     def _capturing_fetch(captured):
95         def _f(uri, dst=None, *, config=None, overwrite=False):
96             captured["uri"] = uri
97             captured["dst"] = dst
98             captured["config"] = config
99             if dst:
100                 with open(dst, "wb") as fh:
101                     fh.write(b"remote-bytes")
102             return dst or uri
103         return _f
104
105
106     def test_acquire_local_input_remote_fetches_to_scratch(monkeypatch):
107         captured = {}

```

```

108 monkeypatch.setattr(storage, "fetch", _capturing_fetch(captured))
109 out = acquire_local_input("gs://bucket/videos/x.mp4", {"backend": "local", "gcs":
{}})
110 assert captured["uri"] == "gs://bucket/videos/x.mp4" # exact URI routed to
fetch
111 assert os.path.basename(out) == "x.mp4" # named from the ref
112 assert os.path.isfile(out) and open(out, "rb").read() == b"remote-bytes"
113 # The whole storage settings dict is forwarded so per-backend (gcs/s3) config
reaches fetch.
114 assert captured["config"] == {"backend": "local", "gcs": {}}
115
116
117 def test_acquire_local_input_input_root_joins_then_fetches(monkeypatch):
118     captured = {}
119     monkeypatch.setattr(storage, "fetch", _capturing_fetch(captured))
120     acquire_local_input("x.mp4", {"input_root": "gs://bucket/videos"})
121     assert captured["uri"] == "gs://bucket/videos/x.mp4" # resolved BEFORE
fetch
122
123
124 def test_acquire_local_input_never_fetches_for_local(monkeypatch):
125     def _boom(*a, **k):
126         raise AssertionError("fetch must not be called for a local input")
127     monkeypatch.setattr(storage, "fetch", _boom)
128     assert acquire_local_input("/data/x.mp4") == "/data/x.mp4"
129     assert acquire_local_input("clip.mp4", {"input_backend": "local", "input_root": ""})
== "clip.mp4"
130

```

19. assistant (2026-06-21T07:57:30.883Z)

Now let me examine the security-critical questions. The key concern: in HOSTED mode (`allowed_video_root`` set), does `validate_input_path`` correctly reject a non-local URI? Let me trace `resolve_under_root`` with a `gs://`` string.

The critical insight: `validate_input_path`` is called with `req.video_path`` (the `**raw**` input), and when `allowed_root`` is set, it calls `resolve_under_root(path, allowed_root)`` which does `os.path.realpath(path)``. Let me verify what `os.path.realpath("gs://bucket/videos/x.mp4")`` does on Windows vs POSIX, and trace the adversarial cases.

20. user (2026-06-21T07:57:35.398Z)

Exit code 1

```

File "<string>", line 14
    'C:..\x',
    ^^^^^^^^^

```

SyntaxError: (unicode error) 'unicodeescape' codec can't decode bytes in position 4-5: truncated \xxx escape

21. user (2026-06-21T07:57:40.607Z)

Exit code 1

```

File "<stdin>", line 13
    'C:..\x',
    ^^^^^^^^^

```

SyntaxError: (unicode error) 'unicodeescape' codec can't decode bytes in position 4-5: truncated \xxx escape

22. assistant (2026-06-21T07:57:43.137Z)

The Windows backslash in the heredoc is being interpreted. Let me write a temp test file instead.

23. assistant (2026-06-21T07:57:51.783Z)

```

import os
from backend.storage import input_is_local
from backend.storage.ref import resolve_input_uri
from backend.utils.paths import validate_input_path, resolve_under_root

print("=== PLATFORM:", os.name, "cwd:", os.getcwd())
print()

```

Hosted mode: allowed_root set (single-tenant containment)

```

ALLOWED = os.path.realpath("/srv/videos")
print("ALLOWED ROOT realpath:", ALLOWED)
print()

```

```

cases = [
    "gs://bucket/videos/x.mp4",
    "gs://../../etc/passwd",
    "s3://b../../etc/shadow",
    "gdrive://../../secret",
    "local://../x",
    " gs://x",
    "C:..\x",
    "",
    "https://user:pass@host/x.mp4",
]

```

Default config (input_backend=local, input_root='') -- what /api/process uses

```

print("--- DEFAULT config (input_backend=local, input_root='') ---")
for c in cases:
    is_local = input_is_local(c, None)
    # validate_input_path with allowed_root set (hosted)
    err = None
    try:
        validate_input_path(c, allowed_root=ALLOWED)
    except ValueError as e:
        err = str(e)
    print(repr(c))
    print(" resolve_input_uri:", repr(resolve_input_uri(c)))
    print(" input_is_local:", is_local)
    print(" validate_input_path(allowed_root) ->", "REJECTED: " + err if err else "PASSED (no error)")
    print()

```

Now with a NON-LOCAL input_root configured (a Drive/bucket-sourced hosted deploy)

```

print("--- input_root='gs://bucket/videos' ---")
cfg = {"input_backend": "local", "input_root": "gs://bucket/videos"}
for c in ["x.mp4", "../../etc/passwd", "/abs/x.mp4"]:
    print(repr(c), "-> resolve:", repr(resolve_input_uri(c, input_backend="local",
input_root="gs://bucket/videos")),
        "is_local:", input_is_local(c, cfg))

```

24. user (2026-06-21T07:57:51.828Z)

File created successfully at: C:/Users/avidu/Projects/badminton-highlight-indexer/scratch/_m2_trace.py (file state is current in your context ? no need to Read it back)

25. user (2026-06-21T07:57:56.593Z)

Exit code 1

Traceback (most recent call last):

```

File "C:\Users\avidu\Projects\badminton-highlight-indexer\scratch\_m2_trace.py", line 2, in
<module>
    from backend.storage import input_is_local

```

ModuleNotFoundError: No module named 'backend'

26. user (2026-06-21T07:58:04.584Z)

Exit code 1

=== PLATFORM: nt cwd: C:\Users\avidu\Projects\badminton-highlight-indexer

ALLOWED ROOT realpath: C:\srv\videos

```
--- DEFAULT config (input_backend=local, input_root='') ---
'gs://bucket/videos/x.mp4'
  resolve_input_uri: 'gs://bucket/videos/x.mp4'
  input_is_local: False
  validate_input_path(allowed_root) -> REJECTED: path 'gs://bucket/videos/x.mp4' escapes the
allowed root 'C:\\srv\\videos'

'gs://../../etc/passwd'
  resolve_input_uri: 'gs://../../etc/passwd'
  input_is_local: False
  validate_input_path(allowed_root) -> REJECTED: path 'gs://../../etc/passwd' escapes the
allowed root 'C:\\srv\\videos'

's3://b/../../etc/shadow'
  resolve_input_uri: 's3://b/../../etc/shadow'
  input_is_local: False
  validate_input_path(allowed_root) -> REJECTED: path 's3://b/../../etc/shadow' escapes the
allowed root 'C:\\srv\\videos'

'gdrive://../../secret'
  resolve_input_uri: 'gdrive://../../secret'
  input_is_local: False
  validate_input_path(allowed_root) -> REJECTED: path 'gdrive://../../secret' escapes the
allowed root 'C:\\srv\\videos'

'local://../x'
  resolve_input_uri: 'local://../x'
  input_is_local: True
  validate_input_path(allowed_root) -> REJECTED: path 'local://../x' escapes the allowed root
'C:\\srv\\videos'

' gs://x'
  resolve_input_uri: ' gs://x'
  input_is_local: True
  validate_input_path(allowed_root) -> REJECTED: path ' gs://x' escapes the allowed root
'C:\\srv\\videos'

'C:..\\x'
  resolve_input_uri: 'C:..\\x'
  input_is_local: True
  validate_input_path(allowed_root) -> REJECTED: path 'C:..\\x' escapes the allowed root
'C:\\srv\\videos'

''
  resolve_input_uri: ''
  input_is_local: True
  validate_input_path(allowed_root) -> REJECTED: path '' escapes the allowed root
'C:\\srv\\videos'
```

Traceback (most recent call last):

File "C:\Users\avidu\Projects\badminton-highlight-indexer\scratch_m2_trace.py", line 29, in
<module>

is_local = input_is_local(c, None)

File "C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage__init__.py", line
46, in input_is_local

return parse_uri(uri).backend == "local"

~~~~~^

```
File "C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\ref.py", line 68, in
parse_uri
    raise ValueError(f"unsupported storage URI scheme: {scheme}:// (in {s!r})")
ValueError: unsupported storage URI scheme: https:// (in 'https://user:pass@host/x.mp4')
```

## 27. assistant (2026-06-21T07:58:17.854Z)

This is a very interesting and important finding. Several things to unpack here. Let me note the key observations:

1. **In HOSTED mode (allowed\_root set), `validate_input_path` rejects every non-local URI** ? because `resolve_under_root` does `os.path.realpath("gs://...")` which on Windows resolves under cwd, fails commonpath containment ? rejected. This is the **good** path the spec claims. ? (Question b answered: correctly rejected in hosted mode.)

2. **`input_is_local(" gs://x")` returns `True`** (leading-whitespace `gs://` is treated as local because `_SCHEME_RE` requires `^`). On POSIX this matters ? let me check. The leading-whitespace case: `input_is_local` returns True, so in **non-hosted mode** (`allowed_root=None`, the default single-user local mode), it would pass the `os.path.exists(" gs://x")` check (returns False ? 404). Not a security escape on its own, but worth tracing on POSIX.

3. **`input_is_local("https://user:pass@host/x.mp4")` RAISES `ValueError`** ? `parse_uri` raises on unknown scheme `https://`. This is a real bug: `input_is_local` is called in `/api/process` at line 278 (after `validate_input_path`) and in the CLI. An unhandled `ValueError` would crash with a 500, not a clean 400. Let me check whether `validate_input_path` would have caught `https://...` first in hosted mode (yes, rejected as out-of-root ? 400) but in **non-hosted mode** `validate_input_path` passes (no null byte, no allowed\_root), then `input_is_local` raises `ValueError` uncaught ? 500. Let me confirm.

Let me now trace POSIX semantics (the actual deployment target is Linux/GCP) and the `https://` crash path precisely.

## 28. assistant (2026-06-21T07:58:25.589Z)

```
import os, posixpath, ntpath
from backend.storage.ref import resolve_input_uri, parse_uri
from backend.utils.paths import validate_input_path
```

## Simulate POSIX semantics for `resolve_under_root` (deployment = Linux/GCP).

```
def posix_resolve_under_root(path, root, cwd="/srv/app"):
    # mimic os.path.realpath on POSIX: join cwd if relative, normalize
    def realpath(p):
        if not posixpath.isabs(p):
            p = posixpath.join(cwd, p)
        return posixpath.normpath(p)
    rp = realpath(path); rr = realpath(root)
    try:
        cp = posixpath.commonpath([rp, rr])
    except ValueError as e:
        return None, "ValueError:" + str(e)
    return (cp == rr), rp

ALLOWED = "/srv/videos"
print("=== POSIX-simulated hosted mode (allowed_root=/srv/videos, cwd=/srv/app) ===")
for c in ["gs://bucket/videos/x.mp4", "gs://.../etc/passwd", " gs://x",
          "https://user:pass@host/x.mp4", "local:///x"]:
    contained, rp = posix_resolve_under_root(c, ALLOWED)
    print(repr(c), "-> realpath", repr(rp), "| contained:", contained)

print()
print("=== input_is_local crash surface (parse_uri raises on unknown scheme) ===")
for c in ["https://h/x", "ftp://h/x", "file:///etc/passwd", "javascript://x",
          "data://x", "rtsp://cam/stream"]:
```

```

try:
    from backend.storage import input_is_local
    r = input_is_local(c, None)
    print(repr(c), "-> input_is_local:", r)
except Exception as e:
    print(repr(c), "-> RAISES", type(e).__name__ + ":", e)

print()
print("=== non-hosted (allowed_root=None) validate_input_path then input_is_local ===")
for c in ["https://user:pass@host/x.mp4"]:
    try:
        validate_input_path(c, allowed_root=None)
        print(repr(c), "validate_input_path: PASSED")
    except ValueError as e:
        print(repr(c), "validate_input_path REJECTED:", e)

```

## 29. user (2026-06-21T07:58:25.604Z)

File created successfully at: C:/Users/avidu/Projects/badminton-highlight-indexer/scratch/\_m2\_trace2.py (file state is current in your context ? no need to Read it back)

## 30. user (2026-06-21T07:58:28.161Z)

```

=== POSIX-simulated hosted mode (allowed_root=/srv/videos, cwd=/srv/app) ===
'gs://bucket/videos/x.mp4' -> realpath '/srv/app/gs://bucket/videos/x.mp4' | contained: False
'gs://.../etc/passwd' -> realpath '/srv/etc/passwd' | contained: False
' gs://x' -> realpath '/srv/app/ gs://x' | contained: False
'https://user:pass@host/x.mp4' -> realpath '/srv/app/https://user:pass@host/x.mp4' | contained:
False
'local://.../x' -> realpath '/srv/app/x' | contained: False

=== input_is_local crash surface (parse_uri raises on unknown scheme) ===
'https://h/x' -> RAISES ValueError: unsupported storage URI scheme: https:// (in 'https://h/x')
'ftp://h/x' -> RAISES ValueError: unsupported storage URI scheme: ftp:// (in 'ftp://h/x')
'file:///etc/passwd' -> RAISES ValueError: unsupported storage URI scheme: file:// (in
'file:///etc/passwd')
'javascript://x' -> RAISES ValueError: unsupported storage URI scheme: javascript:// (in
'javascript://x')
'data://x' -> RAISES ValueError: unsupported storage URI scheme: data:// (in 'data://x')
'rtsp://cam/stream' -> RAISES ValueError: unsupported storage URI scheme: rtsp:// (in
'rtsp://cam/stream')

=== non-hosted (allowed_root=None) validate_input_path then input_is_local ===
'https://user:pass@host/x.mp4' validate_input_path: PASSED

```

## 31. assistant (2026-06-21T07:58:31.452Z)

This confirms the key findings. Let me now verify the exact crash path in `/api/process` and the CLI ? confirming `input\_is\_local` raising `ValueError` is NOT caught. Let me look at the exact ordering in main.py once more and check whether there's a try/except around the `input\_is\_local` call.

## 32. user (2026-06-21T07:58:32.435Z)

```

269         allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
270         try:
271             validate_input_path(req.video_path, allowed_root=allowed_root)
272         except ValueError as e:
273             raise HTTPException(status_code=400, detail=str(e)) from e
274         # M2: the local-existence fast-fail only applies to a LOCAL input. A bucket/Drive
URI can't be
275         # cheaply os.path.exists() 'd ? its existence is verified when the worker fetches it
(a fetch miss

```

```

276         # fails the job loudly). validate_input_path above still runs for every input: its
null-byte guard
277         # always applies, and a configured allowed_video_root (hosted mode) rejects a non-
local URI.
278         if storage.input_is_local(req.video_path, getattr(global_config, "storage", None)):
279             if not os.path.exists(req.video_path):
280                 raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
281         if req.segmenter and not segmenter_registry.get(req.segmenter):
282             available = list(segmenter_registry.list_available().keys())
283             raise HTTPException(
284                 status_code=400,
285                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}")
286         )
287
288         job_id = uuid.uuid4().hex
289         if not db_instance.create_job(job_id, req.video_path, req.segmenter,
290                                     req.player_pool, req.max_frames):

```

### 33. assistant (2026-06-21T07:58:37.474Z)

Confirmed. The `storage.input_is_local(...)` call at line 278 is **outside** the `try/except ValueError` block (which only wraps `validate_input_path` at lines 270-273). So for an unknown-scheme URI like `https://...`, `ftp://...`, `file://...` in the **default/non-hosted config** (`allowed_video_root` unset ? which is the documented default for single-user local mode), `validate_input_path` passes, then `input_is_local` raises an uncaught `ValueError` ? FastAPI returns **HTTP 500** instead of a clean 400.

Now let me assess severity. Is this a security issue or a robustness/error-handling issue? Let me check the `presets.py` / loader to confirm `allowed_video_root` is unset by default (so the non-hosted path is the default).

### 34. user (2026-06-21T07:58:39.824Z)

```

SecurityConfig.allowed_video_root default: None
AppConfig.security.allowed_video_root: None

```

### 35. assistant (2026-06-21T07:58:42.527Z)

Confirmed: default `allowed_video_root=None` (non-hosted). So the 500-crash path is the **default** deployment. Let me verify this end-to-end through the actual FastAPI endpoint with a `TestClient` to be 100% sure it's a 500 and not handled by a global exception handler.

### 36. user (2026-06-21T07:58:52.858Z)

```

13:28:52 [INFO] httpx: HTTP Request: POST http://testserver/api/process "HTTP/1.1 500 Internal
Server Error"
13:28:52 [INFO] httpx: HTTP Request: POST http://testserver/api/process "HTTP/1.1 500 Internal
Server Error"
13:28:52 [INFO] httpx: HTTP Request: POST http://testserver/api/process "HTTP/1.1 500 Internal
Server Error"
13:28:52 [INFO] httpx: HTTP Request: POST http://testserver/api/process "HTTP/1.1 404 Not Found"
'https://user:pass@host/x.mp4' -> 500
'ftp://h/x.mp4' -> 500
'file:///etc/passwd' -> 500
'nonexistent_local.mp4' -> 404 (expected 404)

```

### 37. assistant (2026-06-21T07:58:58.001Z)

Confirmed: unknown-scheme URIs return **HTTP 500** (uncaught `ValueError` from `parse_uri`) instead of a clean 400. This is the most concrete finding ? a real regression in the API contract introduced by M2 wiring `input_is_local` outside the `try/except`.

Now let me address the remaining lens questions thoroughly:

**(d) SSRF / unexpected egress** ? Can a user URI cause an unintended network fetch on a deployment that didn't opt in? Let me trace `acquire_local_input`` for a `gs://`` URI in the **default config**. The key: `acquire_local_input`` resolves the URI; if the resolved `ref.backend != "local"`, it calls `fetch(uri, ...)`` regardless of `input_backend`` config. The default config does NOT gate this. Let me verify.

### 38. user (2026-06-21T07:59:02.047Z)

```
acquire_local_input('gs://...', default-cfg) -> RuntimeError: NETWORK FETCH ATTEMPTED to
gs://attacker-bucket/x.mp4
fetch called with: ['gs://attacker-bucket/x.mp4']
```

### 39. assistant (2026-06-21T07:59:10.964Z)

Important: **even under the DEFAULT config that did NOT opt into remote input**, a user-supplied `gs:///`s3:///`gdrive:///`` URI in `video_path`` triggers a remote fetch via `acquire_local_input`` ? `storage.fetch``. The config flags `input_backend`/`input_root`` gate how **bare** names resolve, but an **explicit scheme always wins** (precedence rule 1) and is fetched regardless. There is no config gate that says "this deployment only accepts local inputs."

However, I need to assess whether this is reachable as an SSRF in the **default** (non-hosted) deployment. Let me trace the full request flow: in `/api/process``, for `gs:///``:

- `validate_input_path`` (non-hosted, `allowed_root=None`): passes (no null byte).
- `input_is_local("gs:///`")`` ? False ? existence check skipped.
- Job is created with `req.video_path = "gs:///`"`, the worker later fetches it.

So in the **default single-user local deployment**, a `gs:///`` URI submitted to `/api/process`` is accepted and queued for a remote fetch. Whether that's "SSRF" depends on the threat model (single-user local API has no untrusted clients), but it IS unexpected egress with no opt-in gate. Let me check the GCS/S3 fetch path to confirm it would actually attempt network egress (not fail closed) and whether `gdrive:///`` reads an arbitrary local mount path (a more concrete local-file-read concern). Let me check `gdrive.py``.

### 40. user (2026-06-21T07:59:11.721Z)

```
1      """Google Drive backend ? reads Drive files through a locally mounted Google Drive.
2
3      We make a deliberate, baked-in assumption: to use Google Drive files you mount Google
4      Drive
5      for Desktop, which exposes Drive as an ordinary filesystem path. That's a fair
6      requirement ? it
7      needs no service account, no API quota, no OAuth, and Drive streams + caches bytes on
8      access. So
9      ``gdrive:///` is just a portable, OS-agnostic alias over that mount: it names a path
10     relative to
11     your "My Drive" folder, and everything else is plain local-filesystem I/O (the same
12     code the
13     ``local`` backend uses).
14
15     gdrive://Sharing/clip.mp4 -> <mount>/My Drive/Sharing/clip.mp4
16
17     Why an alias instead of just a raw path? Portability: the mount root differs per machine
18     (Windows ``G:\My Drive``, macOS ``~/Library/CloudStorage/GoogleDrive-<acct>/My
19     Drive``), so the same
20     ``gdrive:///` URI resolves on any box with Drive mounted. The root is auto-detected,
21     or set it
22     explicitly via ``GDRIVE_MOUNT_ROOT`` / ``storage.gdrive.mount_root`` (a path, not a
23     secret).
24
25     A headless box (a GPU VM, CI) has no mount ? there, stage the file to a bucket and
26     use
27     ``gcs:///` / ``s3:///`, or ``gdown <file-id>`` to a local path. This backend
```

intentionally does NOT

```
19     talk to the Drive API. When the mount (or a file in it) is missing, errors point the
user at the
20     fix: install Drive for Desktop and include the folder in syncing so it can be pulled
locally.
21     """
22     from __future__ import annotations
23
24     import glob
25     import os
26     import string
27     from typing import Iterator, Optional
28
29     from backend.storage.base import StorageBackend
30     from backend.storage.local import LocalStorage
31     from backend.storage.ref import StorageRef, StorageStat
32
33     _INSTALL_URL = "https://www.google.com/drive/download/"
34
35     # Shared remedy line ? how to make a folder available locally. Drive for Desktop streams
files,
36     # but a folder only appears once it's part of your synced Drive (and a *shared* folder
must first
37     # be added to "My Drive"). The same fix resolves both "not mounted" and "mounted but
missing".
38     _SYNC_REMEDY = (
39         "make sure the folder is INCLUDED in syncing so its files can be pulled locally: in
Google "
40         "Drive for Desktop keep 'Stream files' on, and for a folder shared with you, open
Drive in the "
41         "browser, right-click it -> 'Add shortcut to Drive' so it appears under 'My Drive'.
Drive then "
42         "downloads the bytes on first access."
43     )
44
45
46     def _not_mounted_msg(tried: Optional[str]) -> str:
47         return (
48             "Google Drive is not mounted, so the gdrive:// backend cannot resolve any
path.\n"
49             f"  Remedy:\n"
50             f"  1. Install Google Drive for Desktop and sign in: {_INSTALL_URL}\n"
51             f"  2. Then {_SYNC_REMEDY}\n"
52             f"  3. If your 'My Drive' is not at the default location (Windows G:\\My Drive,
macOS "
53             f"~/Library/CloudStorage/GoogleDrive-<acct>/My Drive), set GDRIVE_MOUNT_ROOT or "
54             f"storage.gdrive.mount_root to point at it (auto-detect tried: {tried!r})."
55         )
56
57
58     def _not_synced_msg(root: str, rel: str, uri: str) -> str:
59         return (
60             f"Google Drive is mounted ({root!r}) but {uri!r} is not present locally "
61             f"(expected at {os.path.join(root, *rel.split('/')})!r}).\n"
62             f"  This usually means the folder is not synced yet ? {_SYNC_REMEDY}\n"
63             f"  (Install/repair Drive for Desktop if needed: {_INSTALL_URL})"
64         )
65
66
67     def _autodetect_mount_root() -> Optional[str]:
68         """Find a mounted Google Drive 'My Drive' folder, or ``None``. Covers the Google
Drive for
69         Desktop defaults: macOS ``~/Library/CloudStorage/GoogleDrive-<acct>/My Drive`` and
Windows
70         ``<letter>:\My Drive`` (``G:`` is the install default). Linux/other ? configure
```

```

explicitly."""
71     home = os.path.expanduser("~")
72     for p in sorted(glob.glob(os.path.join(home, "Library", "CloudStorage",
"GoogleDrive*", "My Drive"))):
73         if os.path.isdir(p):
74             return p
75     if os.name == "nt":
76         # G is the Drive-for-Desktop default; then scan the rest of the assignable
letters.
77         for letter in ["G"] + [c for c in string.ascii_uppercase if c != "G"]:
78             p = f"{letter}:\My Drive"
79             if os.path.isdir(p):
80                 return p
81     return None
82
83
84     class GDriveStorage(StorageBackend):
85         def __init__(self, *, mount_root: Optional[str] = None, **_ignored) -> None:
86             root = mount_root or os.environ.get("GDRIVE_MOUNT_ROOT") or
_autodetect_mount_root()
87             if not root or not os.path.isdir(root):
88                 raise RuntimeError(_not_mounted_msg(root))
89             self._root = os.path.abspath(root)
90             self._local = LocalStorage(base_dir=self._root) # all I/O is plain local-FS
under the mount
91
92         def _local_ref(self, ref: StorageRef) -> StorageRef:
93             """Translate a ``gdrive://<rel>`` ref into a ``local`` ref keyed by an OS path
relative to
94             the mount root (``LocalStorage`` joins it under ``base_dir``)."""
95             rel = ref.key.lstrip("/")
96             key = os.path.join(*rel.split("/")) if rel else ""
97             return StorageRef("local", "", key, f"local://{key}")
98
99         def _require_present(self, ref: StorageRef) -> None:
100             """Guard the read paths: a path missing from the mount almost always means the
folder
101             isn't synced ? raise with that remedy instead of a cryptic FileNotFoundError
downstream."""
102             rel = ref.key.lstrip("/")
103             abs_path = os.path.join(self._root, *rel.split("/")) if rel else self._root
104             if not os.path.exists(abs_path):
105                 raise FileNotFoundError(_not_synced_msg(self._root, rel, ref.uri))
106
107         def _to_gdrive_uri(self, abs_path: str) -> str:
108             """The inverse: an absolute path under the mount ? its ``gdrive://`` URI."""
109             rel = os.path.relpath(abs_path, self._root).replace(os.sep, "/")
110             return f"gdrive://{rel}"
111
112         def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
113             # The file is already local (via the mount): no dst => identity passthrough to
the mount
114             # path; a dst => copy out (a byte-stable scratch copy), exactly as the local
backend does.
115             self._require_present(ref)
116             return self._local.fetch_to_local(self._local_ref(ref), dst,
overwrite=overwrite)
117
118         def put(self, local_path, ref, *, content_type=None) -> StorageRef:
119             self._local.put(local_path, self._local_ref(ref), content_type=content_type)
120             return ref
121
122         def exists(self, ref) -> bool:
123             return self._local.exists(self._local_ref(ref))
124

```

```

125     def stat(self, ref) -> StorageStat:
126         self._require_present(ref)
127         return self._local.stat(self._local_ref(ref))
128
129     def list(self, prefix) -> Iterator[StorageRef]:
130         for r in self._local.list(self._local_ref(prefix)):
131             uri = self._to_gdrive_uri(r.key)
132             yield StorageRef("gdrive", "", uri[len("gdrive://"):], uri)
133
134     def delete(self, ref) -> None:
135         self._local.delete(self._local_ref(ref))
136

```

#### 41. assistant (2026-06-21T07:59:18.697Z)

Important traversal observation for `gdrive://`: `\_local\_ref` does `rel = ref.key.lstrip("/")` then `os.path.join(\*rel.split("/"))`. For `gdrive://.../etc/passwd`, `ref.key = ".../etc/passwd"`, `rel.split("/")` = `["..", "..", "etc", "passwd"]`, `os.path.join` keeps the `..` segments, then `LocalStorage.\_path` joins under `base\_dir` (the mount root) WITHOUT containment check ? \*\*path traversal escaping the Drive mount\*\*. But this is pre-existing gdrive backend behavior, not introduced by M2. The M2 change is that `acquire\_local\_input` now routes `gdrive://` URIs from the API/CLI `video\_path` into this backend. Whether this is in-scope: M2 wires the input, so it newly \*exposes\* this. But it requires `gdrive` mount configured. Let me note it but keep it lower confidence/severity as it's a pre-existing backend gap now reachable.

Let me now look at the 3 new tests in test\_api\_endpoints.py to understand what the author tested, and confirm they don't cover the unknown-scheme 500.

#### 42. user (2026-06-21T07:59:19.345Z)

```

diff --git a/tests/test_api_endpoints.py b/tests/test_api_endpoints.py
index 001c9fc..ebedddf 100644
--- a/tests/test_api_endpoints.py
+++ b/tests/test_api_endpoints.py
@@ -334,3 +334,61 @@ def test_parity_job_worker_vs_direct_cli_path(env, tmp_path, monkeypatch):
     jobf = client.get(f"/api/jobs/{rf.json()['job_id']}").json()
     assert jobf["result"]["status"] == "failed"
     # empty-run similar (would preserve prior via reingest safety in both paths)
+
+
+## ----- M2: URI-aware input
+class _CaptureSeg:
+    """Records the video_path the segmenter actually received (M2: must be a LOCAL path)."""
+    last_path = None
+
+    def __init__(self, db, cfg):
+        self.db = db
+
+    def process_video(self, video_id, video_path, *a, **k):
+        _CaptureSeg.last_path = video_path
+        sid = self.db.add_play_segment(video_id, 0.0, 5.0)
+        return [{"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0, "metadata":
+{}}]
+
+def test_process_local_input_passes_identity_path(env):
+    """Default (local) input must reach the segmenter as the EXACT same path ? identity, no
+copy
+    (the M2 zero-behaviour-change guarantee for today's local workflow)."""
+    client, db, tmp_path, mp = env
+    vid = _video(tmp_path)
+    mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
+    mp.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)
+    _CaptureSeg.last_path = None
+    r = client.post("/api/process", json={"video_path": str(vid), "segmenter": "x"})

```

```

+   assert r.status_code == 202
+   assert _CaptureSeg.last_path == str(vid)
+
+
+def test_process_local_missing_file_still_404(env):
+   """A LOCAL input that doesn't exist still fast-fails 404 ? the existence gate is preserved
+   for local paths (only non-local URIs skip it)."""
+   client = env[0]
+   r = client.post("/api/process", json={"video_path": "/no/such/local/file.mp4"})
+   assert r.status_code == 404
+
+
+def test_process_nonlocal_uri_skips_exists_and_fetches(env):
+   """A gs:// input is NOT 404'd at submit (can't os.path.exists a bucket object); the worker
+   fetches it to a local scratch file and the segmenter sees that LOCAL path, not the URI."""
+   client, db, tmp_path, mp = env
+
+   def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
+       with open(dst, "wb") as fh:
+           fh.write(b"fetches-video-bytes")
+       return dst
+
+   mp.setattr(m.storage, "fetch", fake_fetch)
+   mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
+   mp.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)
+   _CaptureSeg.last_path = None
+   r = client.post("/api/process", json={"video_path": "gs://bucket/videos/x.mp4",
"segmenter": "x"})
+   assert r.status_code == 202                                # NOT 404 ? non-local skips the
exists gate
+   job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
+   assert job["status"] == "done"
+   assert _CaptureSeg.last_path is not None
+   assert not _CaptureSeg.last_path.startswith("gs://")      # a fetched LOCAL path, not the
URI
+   assert os.path.basename(_CaptureSeg.last_path) == "x.mp4"

```

#### 43. assistant (2026-06-21T07:59:26.633Z)

The tests cover the happy paths (local identity, local 404, gs:// fetch) but NOT the unknown-scheme `https://`/`ftp://`/`file://` case ? which 500s. That's the gap.

Now let me verify one more thing for completeness: the byte-identity guarantee (M2's core SAFE requirement). For the default config, does `acquire\_local\_input` return the path byte-identically? The test confirms `acquire\_local\_input(str(p)) == str(p)`. But there's a subtle case ? `local://` prefixed paths. The worker uses bare paths. Let me confirm a bare local path stays identical (already tested) and check whether the DB record / report keeps the original URI as the spec requires.

Looking at main.py line 251: `emit\_indexer\_report(... video\_path=req.video\_path ...)` ? uses original URI ?. And `db\_instance.create\_job(job\_id, req.video\_path, ...)` line 289 ? original URI ?. Good, that matches spec.

Let me run the actual new tests plus the existing API/storage suites to confirm green, and check for any lint concern with the unused `posixpath` import etc. (in my scratch, not repo).

#### 44. user (2026-06-21T07:59:40.803Z)

```

..... [100%]
42 passed in 11.11s

```

#### 45. assistant (2026-06-21T07:59:47.444Z)

All 42 pass. Now let me double-check the `gdrive://` traversal claim concretely (is it newly

reachable and does it actually escape), and confirm whether `acquire\_local\_input` for a `local://../x` resolves to a traversal too ? since `input\_is\_local("local://../x")` returned True, it would take the identity branch returning `ref.key = "../x"` which then is read in place relative to cwd. Let me verify the `local://` identity-branch key value.

#### 46. user (2026-06-21T07:59:50.707Z)

```
'local://../etc/passwd' is_local: True -> ref.key: '../etc/passwd' | acquire returns:
'../etc/passwd'
'local:///etc/passwd' is_local: True -> ref.key: '/etc/passwd' | acquire returns: '/etc/passwd'
'local://rel/x.mp4' is_local: True -> ref.key: 'rel/x.mp4' | acquire returns: 'rel/x.mp4'
```

#### 47. assistant (2026-06-21T07:59:57.323Z)

This is a significant containment finding. With `local://` prefix:  
- `validate\_input\_path("local://../etc/passwd", allowed\_root=ALLOWED)` ? runs  
`resolve\_under\_root("local://../etc/passwd", ALLOWED)` ?  
`os.path.realpath("local://../etc/passwd")` ? resolves `local://` as a dir under cwd, then  
`..` escapes ? NOT contained ? rejected. So in HOSTED mode the \*raw string\* `local://...` is  
checked and rejected (good).

BUT there's a mismatch: `validate\_input\_path` checks the \*\*raw\*\* `local://../etc/passwd`  
string (with the `local://` literal prefix in the path), whereas `acquire\_local\_input` strips  
the prefix and returns `../etc/passwd` (the real path read by the segmenter). The containment  
check validates a \*different string\* than what's actually opened. Let me verify whether this  
mismatch could let a path that the segmenter reads escape the root while the raw-string check  
passes ? i.e., a TOCTOU/normalization mismatch. Let me test a crafted `local://` that passes  
`validate\_input\_path` on the raw string but resolves (after stripping) to an out-of-root file.

#### 48. user (2026-06-21T08:00:06.629Z)

ALLOWED: C:\Users\avidu\Projects\badminton-highlight-indexer\videos

```
'local://videos/../etc/passwd'
  raw passes validate_input_path: False
  segmenter opens: 'videos/../etc/passwd' -> real: C:\Users\avidu\Projects\etc\passwd
  stripped contained in ALLOWED: False
  *** MISMATCH (raw passes but stripped escapes): False

'local://../etc/passwd'
  raw passes validate_input_path: False
  segmenter opens: '../etc/passwd' -> real: C:\Users\avidu\Projects\etc\passwd
  stripped contained in ALLOWED: False
  *** MISMATCH (raw passes but stripped escapes): False

'local://./local:../etc/passwd'
  raw passes validate_input_path: False
  segmenter opens: './local:../etc/passwd' -> real: C:\Users\avidu\Projects\badminton-
highlight-indexer\etc\passwd
  stripped contained in ALLOWED: False
  *** MISMATCH (raw passes but stripped escapes): False
```

#### 49. assistant (2026-06-21T08:00:13.125Z)

In these cases both reject (no exploitable mismatch found here) ? the `local://` prefix adds a  
`local:` path segment that makes the raw realpath land \*deeper\*, so traversal needs one more  
`.` and both strings escape together. I couldn't construct a clean raw-passes-but-stripped-  
escapes mismatch, so I won't claim a containment bypass via `local://`. The conceptual concern  
(validate checks raw, acquire opens stripped) is real but I have no working exploit, so it's a  
low-confidence robustness note, not a finding.

Let me do a final check: confirm the existing full storage + paths test suites stay green (the  
SAFE/default-OFF guarantee), and check whether there's a global exception handler that might  
already convert the `ValueError` 500 into something else in production (I tested with TestClient

default ? but let me check if the app registers one).

**50. user (2026-06-21T08:00:14.737Z)**

No matches found

**51. user (2026-06-21T08:00:16.582Z)**

ERROR: file or directory not found: tests/test\_storage\_ref.py

no tests ran in 0.00s

**52. assistant (2026-06-21T08:00:24.401Z)**

No global exception handler exists, so the 500 stands. Let me find the actual storage/paths test file names and run them.

**53. user (2026-06-21T08:00:25.011Z)**

test\_gemini\_refine.py  
test\_path\_safety.py  
test\_storage.py

**54. user (2026-06-21T08:00:29.365Z)**

..... [100%]  
71 passed in 0.63s

**55. assistant (2026-06-21T08:00:33.950Z)**

All green. Let me clean up my scratch files (gitignored, but tidy anyway) and finalize.

**56. user (2026-06-21T08:00:34.695Z)**

done

**57. assistant (2026-06-21T08:00:45.685Z)**

Let me summarize my findings. I have one high-confidence concrete bug (the 500 crash), one config-gating/egress concern, and one lower-confidence pre-existing-backend traversal note now reachable via M2.

To recap the lens questions:

- **(a)** Null-byte check always runs (good); ``allowed_video_root`` containment in HOSTED mode correctly rejects all non-local URIs (verified: ``gs://``, ``s3://``, ``gdrive://``, ``local://../x`` all rejected because ``os.path.realpath`` of the URI-string + commonpath fails). No bypass of `validate_input_path` found for the adversarial inputs.
- **(b)** Yes ? in hosted mode a non-local URI is correctly rejected (``resolve_under_root`` realpath on the ``gs://..`` string lands under cwd, escapes root ? ValueError ? 400).
- **(c)** ``acquire_local_input`` fetches to ``tempfile.mkdtemp`` with filename = ``ref.name`` (basename only, via ``key.rsplit("/")[-1]``), so a key like ``../../etc/passwd`` yields name ``passwd`` ? no traversal into the scratch write path. Safe.
- **(d)** Real concern: explicit-scheme URIs trigger remote fetch with no config opt-in gate; and unknown schemes 500.

**58. user (2026-06-21T08:01:16.209Z)**

Structured output provided successfully

**59. assistant (2026-06-21T08:01:30.556Z)**

I've completed the security & path-safety review of M2. Summary of what I found and verified:

**\*\*Lens answers:\*\***

- **\*\*(a) Null-byte / containment bypass:\*\*** No bypass. ``validate_input_path``'s null-byte check always runs, and in HOSTED mode (``allowed_video_root`` set) ``resolve_under_root`` realpaths the URI-string and correctly rejects every non-local URI I tried (``gs://``, ``gs://../etc``, ``s3://b/../../../../etc``, ``gdrive://../../../../secret``, ``local://../x``, ``gs://x``, ``C:..\x``, empty).
- **\*\*(b) Non-local URI in hosted mode:\*\*** Correctly rejected (realpath of ``gs://..`` lands under cwd, commonpath fails ? `ValueError` ? 400). Verified.
- **\*\*(c) Scratch-fetch safety:\*\*** Safe. ``acquire_local_input`` fetches to ``tempfile.mkdtemp`` and names the file ``ref.name``, which is a basename-only (``key.rsplit("/")[-1]``), so a key like ``../etc/passwd`` yields filename ``passwd`` ? no traversal into the write path.
- **\*\*(d) SSRF/egress:\*\*** Real gap ? an explicit-scheme URI is fetched regardless of ``input_backend``/``input_root`` config; there is no opt-in gate for remote input.

**\*\*Three findings filed:\*\***

1. **\*\*major / high confidence\*\*** ? ``https://``/``ftp://``/``file://`` in ``video_path`` returns HTTP **\*\*500\*\*** (uncaught ``ValueError`` from ``parse_uri``, because ``input_is_local`` at ``backend/main.py:278`` sits outside the try/except). Reproduced end-to-end via `TestClient`. Pre-M2 these returned a clean 404. The same gap exists in the CLI path.
2. **\*\*minor / high\*\*** ? explicit remote URIs trigger egress with no per-deployment opt-in gate.
3. **\*\*minor / medium\*\*** ? ``gdrive://``/``local://``/``..`` traversal escapes the mount/cwd (pre-existing backend behavior, newly reachable from the request boundary via M2; mitigated in hosted mode by ``validate_input_path``).

The default-OFF / byte-identity guarantee itself holds: local/bare paths are identity passthrough (no copy), the original URI is preserved for the DB record + report, and all 71 storage/path/input tests pass. The one concrete regression is the 500 on unknown schemes.

Relevant files: ``C:/Users/avidu/Projects/badminton-highlight-indexer/backend/main.py``,  
``C:/Users/avidu/Projects/badminton-highlight-indexer/backend/storage/__init__.py``,  
``C:/Users/avidu/Projects/badminton-highlight-indexer/backend/storage/ref.py``,  
``C:/Users/avidu/Projects/badminton-highlight-indexer/backend/storage/gdrive.py``,  
``C:/Users/avidu/Projects/badminton-highlight-indexer/backend/utils/paths.py``.